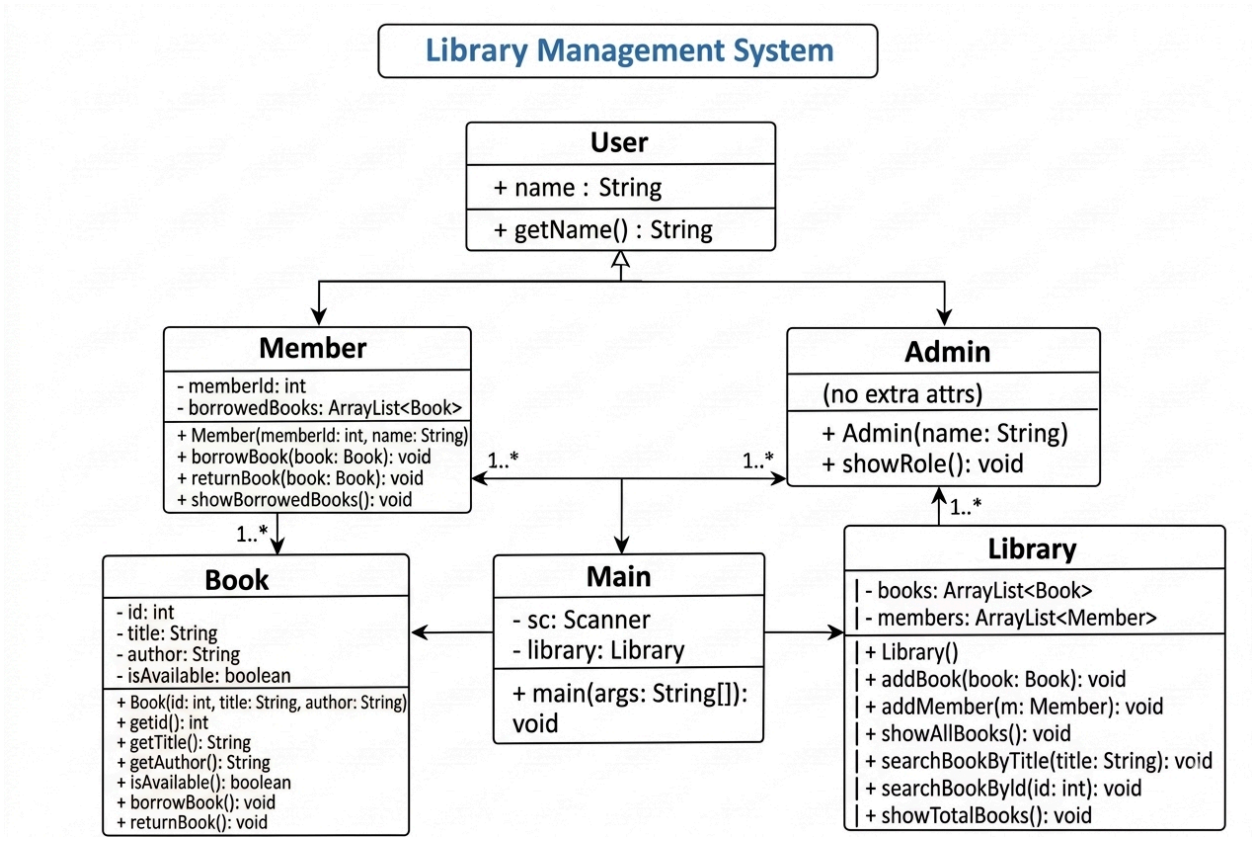
Course Title:Object Oriented Programming
Project Title: Library Management System

Student Name: Niti Mohajan
Roll Number:61
Instructor Name: Soumadip Biswas
Department : IT
Spring 2026

Problem Statement :

Usually in many small schools and colleges, library records are maintained manually and thats why they suffer from lack of accuracy and are prone to lose books, which makes it very difficult to keep track of issued books. Therefore the purpose of this project is to develop a console based Library Management System in Java, in order to make the management of books easy and efficient. The Library Management System will enable librarians to easily add new books , register new members and then issue and return the books and maintain the records. It will be built using the principles of object oriented programming, in order to create a system that is easy to maintain, efficient and highly modular and also improves accuracy in library works.

System Design :

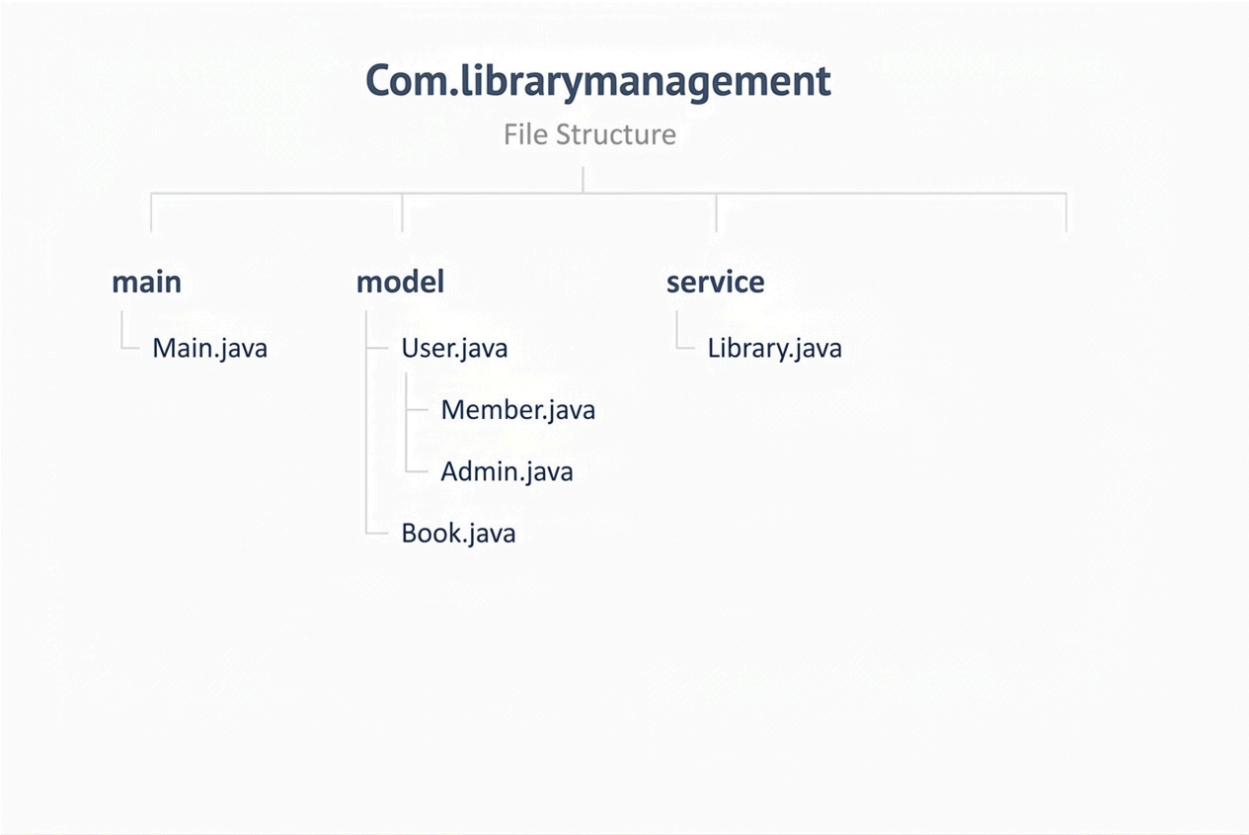


UML Diagram

Description of Major Classes:

Library: Manages collections of books, members, and transactions.
Book: Stores book details such as bookId, title, author, and isAvailable.
Member: Represents library users with memberId, name, and a borrowed books list.
Admin: Inherits Member. Can add/remove books and override borrow rules.

Package structure



Interaction Flow:

1.Program starts in LibraryManagementSystem.java.
2.User chooses an action: add/search/display/borrow/return books.
3.Library class processes requests.
4.Book and Member objects are updated accordingly.
5.System shows success/error messages based on action outcome.

OOP Concepts Demonstrated:

Concept	Where Used	Why Used	Code Snippet	Explanation
Encapsulation	Book class	Protect book details	java class Book { private String title; public String getTitle() { return title; } }	Title is private, accessed only via getter method to prevent accidental changes.
Inheritance	Admin extends Member	Admin has all member functionality plus	java class Admin extends Member { void	Avoids code duplication, Admin is a

		extra	addBook(Book b) { ... } }	specialized Member.
Polymorphism	borrowBook() override	Different borrow behavior for Admin vs Member	java @Override void borrowBook(Book b) { ... } .	Same method name, different behavior implementation.
Abstraction	Library interface	Hide internal logic from user	java interface LibraryOperations { void addBook(Book b); void removeBook(int id); }	Users interact with simple methods, internal logic hidden.

Implementation Highlights:

Snippet 1 – Adding a Book

```
public void addBook(Book book) {
   books.add(book);
   System.out.println("Book added: " + book.getTitle());
}
```

Justification: Shows Admin only functionality and object addition.

Snippet 2 – Borrowing a Book

```
public void borrowBook(int bookId, Member m) {
   Book b = findBook(bookId);
   if (b.isAvailable()) {
      b.setAvailable(false);
      m.addBorrowedBook(b);
   } else {
      System.out.println("Book not available");
   }
}
```

Justification: Demonstrates interaction between objects and encapsulation.

Snippet 3 – Displaying All Books

```
public void displayBooks() {
   for(Book b : books) {
      System.out.println(b.getTitle() + " by " + b.getAuthor());
   }
}
```

Justification: Iterates through objects and displays book details.

Snippet 4 – Returning a Book

```
public void returnBook(int bookId, Member m) {
    Book b = findBook(bookId);
    if(m.hasBorrowed(b)) {
        b.setAvailable(true);
        m.removeBorrowedBook(b);
        System.out.println("Book returned successfully");
    }
}
```

Justification: Handles return logic and validates borrowed books list.

Testing & Error Handling:

Test Cases:

Borrowing an already borrowed book.
Returning a book not borrowed.
Admin adding duplicate books.
Searching for non-existent books.

Edge Cases:

Empty book list.
Invalid member ID.
Null book input.

Failure Scenarios:

Borrowing unavailable book -System displays “Book not available.”
Returning book not borrowed -System shows error message.

Git Workflow:

← **Commits**

add available books feature and upda... 8d
nitimohajan authored

fix Book class methods for borrow an... 8d
nitimohajan authored

improve returnBook with validation an... 8d
nitimohajan authored

improve borrowBook method with me... 8d
nitimohajan authored

add complete Library Management Sy... 8d
nitimohajan authored

add Library class for managing books ... 8d
nitimohajan authored

add Member class with borrow/return ... 8d

← **Commits**

complete README with project detail... 3d
nitimohajan authored

update Member and Library to use en... 8d
nitimohajan authored

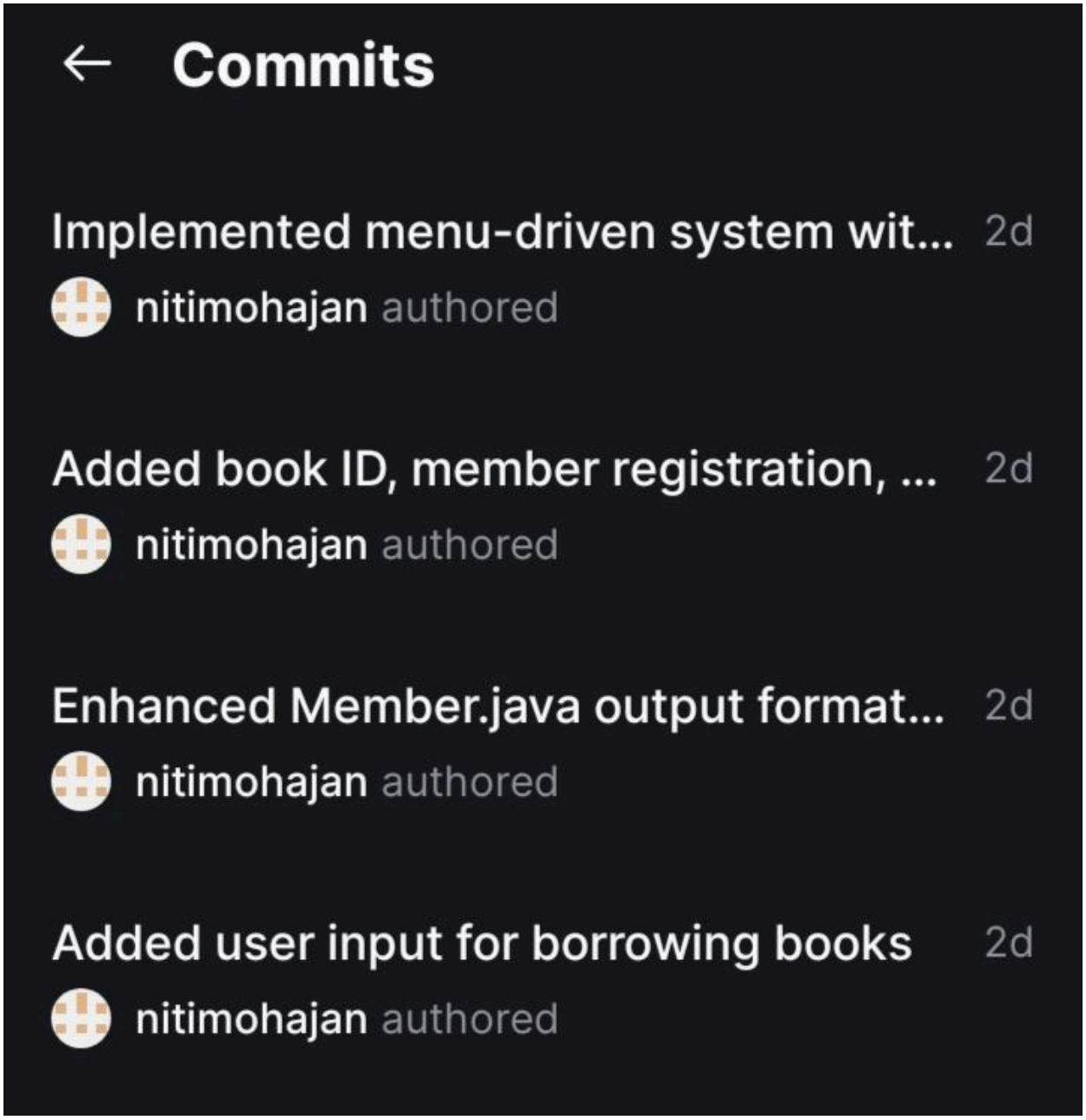
apply encapsulation to Book class wit... 8d
nitimohajan authored

add and integrate Admin class into sy... 8d
nitimohajan authored

add Admin class extending User for p... 8d
nitimohajan authored

add User base class for inheritance 8d
nitimohajan authored

final polish UI and improve system out... 8d
nitimohajan authored



Development Progression:

The development of the Library Management System followed a structured and incremental approach. First, the core classes were created, including Book, Member, Library, User, and Admin, establishing the foundation for the system. Next, essential functionalities such as borrowing and returning books, displaying available books, and tracking the total book count were implemented and refined with proper validations. The classes were then improved using encapsulation and other object oriented principles to enhance code quality and maintainability. After that, all components were integrated into a complete system, and documentation was added to explain its features and usage. Finally, a menu driven interface was developed with improved user input handling and output formatting, providing a smooth and interactive experience for library users.

8. Conclusion & Future Scope:

The Library Management System successfully demonstrates key object-oriented programming concepts such as classes, inheritance, and encapsulation while providing essential functionalities like adding, borrowing, returning, and listing books, along with member and admin management. In the future, the system can be enhanced by integrating a database for persistent storage, adding online catalog search, due date reminders, fine calculation, book recommendations, and mobile support, as well as generating reports and analytics for better library management. Overall, this project provides a modular, maintainable, and user-friendly system, reinforcing programming skills and laying the foundation for more advanced library management solutions.

Copy of the initial project proposal:
Library Management System using Object Oriented Programming in Java

Problem Statement:
Usually in many small schools and colleges, library records are maintained manually and thats why they suffer from lack of accuracy and are prone to lose books, which makes it very difficult to keep track of issued books. Therefore the purpose of this project is to develop a console based Library Management System in Java, in order to make the management of books easy and efficient. The Library Management System will enable librarians to easily add new books , register new members and then issue and return the books and maintain the records. It will be built using the principles of object oriented programming, in order to create a system that is easy to maintain, efficient and highly modular and also improves accuracy in library works.

Target User / Audience :
1.School and College Libraries
2.Local Libraries
3.Librarians .
Core Features :
1.It will manage book records in the library 2.It will add and register members
3.It will help in search book by Title/Id
4.It will help to display the available and unavailable books
5.It will help in managing the information of retuning books and update availability
6.It will help in input validation and error handling

OOP Concepts to be Used:
Abstraction
Inheritance
Polymorphism
Encapsulation
Exception Handling
Collections

Brief Architecture Description:
The system will be divided into separate classes such as Book, Member, and Library to maintain clear separation of responsibilities and it will use four principles of OOP such as Encapsulation,Inheritance, Polymorphism and Abstraction.Here,the book and member classes will store relevant data, while the library class will manage operations such as adding books, issuing books, and returning books using collections.An abstract base class will define the common member attributes and different member types will extend it to promote code reuse and structured hierarchy.Polymorphism will allow varying borrowing limits for different member categories. Data will be secured through encapsulation, and exception handling will ensure the system manages invalid inputs and prevent invalid operations.The main class will provide a console based interface for user interaction.

Copy of the review mail receied (with APPROVED_MINOR):

PCCCS495 Proposal Review – APP...

S

soumadip.biswas@ieminternat... 2 Mar

To You

Dear Niti Mohajan ,

Project: Library Management System using Object Oriented Programming in Java
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal presents a well-defined problem statement, clear objectives, and a comprehensive list of core features for a console-based Library Management System. It effectively identifies target users and outlines a practical scope for a university OOP project. A significant strength is the detailed explanation of how OOP principles (Encapsulation, Inheritance, Polymorphism, and Abstraction) will be applied